



线程池的关闭

北京理工大学计算机学院
金旭亮

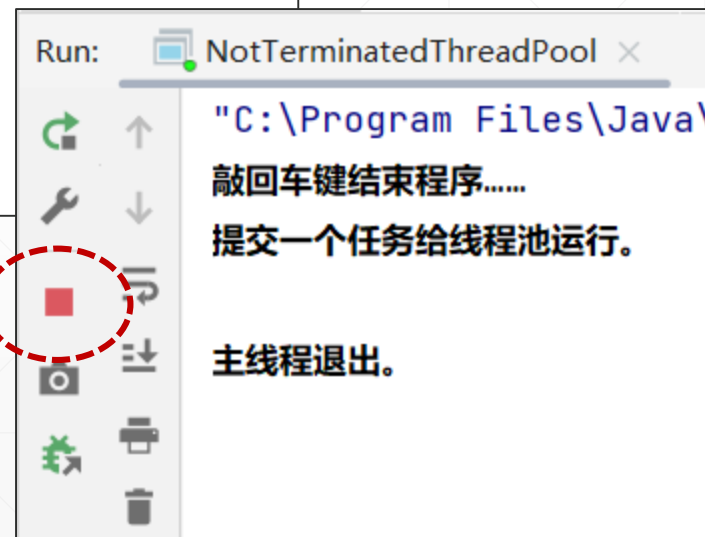
引例

NotTerminatedThreadPool.java

```
public class NotTerminatedThreadPool {  
    public static void main(String[] args) {  
        var threadPool : ExecutorService = Executors.newFixedThreadPool( nThreads: 1);  
        threadPool.submit(() -> {  
            System.out.println("提交一个任务给线程池运行。");  
        });  
        System.out.println("敲回车键结束程序.....");  
        new Scanner(System.in).nextLine();  
        System.out.println("主线程退出。");  
    }  
}
```



为什么主线程退出了，程序却仍然没有结束？



引例解析



与普通的线程不一样，线程池被设计成“长期运行的”。



即使任务队列空了，线程池也不会自动关闭，而是在那里“时刻准备着”，随时完成用户提交的各种工作任务。



线程池不会“自动关闭”的这个特性，可能会带来问题，比如，由于它的运行，会导致进程长久不能结束，而我们可能原先希望线程池干完活以后就自动关闭本程序和释放所有资源。



怎样关闭线程池？

线程池关闭示例-1

为了展示关闭线程池的基本方法，设计了一个示例，右图所示为将被线程池执行的一个示例任务。

```
//一个将要被线程池执行的任务
2 usages
class ShutDownTask implements Runnable {
    2 usages
    private String taskInfo = "";
    2 usages
    public ShutDownTask(String info) {
        taskInfo = info;
    }
    @Override
    public void run() {
        var threadName : String = Thread.currentThread().getName();
        try {
            Thread.sleep( millis: 1000);
            System.out.println(threadName + "处理: " + taskInfo);
        } catch (InterruptedException e) {
            System.out.println(threadName + "被中断了");
        }
    }
}
```

ShutDown.java

线程池关闭示例-2



```
public class ShutDown {  
    public static void main(String[] args) throws InterruptedException {  
        //创建一个包容5个线程的线程池，向它提交10个任务  
        ExecutorService threadPool = Executors.newFixedThreadPool( nThreads: 5);  
        for (int i = 1; i <= 10; i++) {  
            threadPool.execute(new ShutDownTask( info: "任务" + i));  
        }  
        //等待1.5秒后关闭线程池  
        Thread.sleep( millis: 1500);  
        threadPool.shutdown();  
        //检查线程池状态  
        System.out.println("isShutdown=" + threadPool.isShutdown(););  
        System.out.println("isTerminated=" + threadPool.isTerminated(););  
        try {  
            //关闭之后，再次提交一个新任务  
            threadPool.execute(new ShutDownTask( info: "新任务"));  
        } catch (RejectedExecutionException ex) {  
            System.out.println("\n线程池正在关闭中，拒绝了新任务。 \n");  
        }  
    }  
}
```

线程池定义了一个shutdown方法用于关闭自己，并且提供了isShutdown和isTerminated两个方法查询其当前状态。

ShutDown.java

线程池关闭示例-3

```
Run: ShutDown x
"C:\Program Files\Java\jdk-17\bin
pool-1-thread-2处理: 任务2
pool-1-thread-1处理: 任务1
pool-1-thread-5处理: 任务5
pool-1-thread-4处理: 任务4
pool-1-thread-3处理: 任务3
isShutdown=true
isTerminated=false

线程池正在关闭中, 拒绝了新任务。

pool-1-thread-5处理: 任务8
pool-1-thread-2处理: 任务6
pool-1-thread-1处理: 任务7
pool-1-thread-4处理: 任务9
pool-1-thread-3处理: 任务10

Process finished with exit code 0
```

调用shutdown方法之后, 线程池并不会立即 (马上) 关闭, 而是需要等一段时间完成收尾工作, 因此, isShutdown()方法返回true, 表明正处于关闭阶段, 但isTerminated()方法会返回false, 因为此时它还没有真的关闭。

发出shutdown命令之后, 线程池不会再接收新的任务, 向它提交新任务, 线程池会抛出异常

虽然发出了shutdown命令, 但已经开始和提交的任务仍然需要运行完毕.....

关闭线程池的方法



shutdown() 方法：不再接收新任务，等待当前任务（包括队列中的剩余任务）执行完毕后再关闭。如果这时再尝试提交任务给线程池，线程池会拒绝接收新任务，并抛出“`RejectedExecutionException`”。



shutdownNow() 方法：立即终止线程池，正在运行的线程被中断，也不再运行队列中的剩余任务，这些未执行的任务打包放到一个 `List<Runnable>` 中返回给外界。



可以使用 **isShutdown() 方法** 来判断是否已经 shutdown 了。



可以使用 **isTerminated() 方法** 来判断是否线程池是不是已经完全停止了。

等待线程池的退出

可以使用`awaitTermination()`方法来判断在指定时间内是否线程池已经完全停止了，注意，执行此方法的线程，是阻塞的。

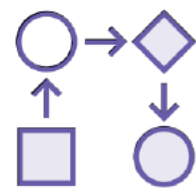
```
public class ScheduleTaskDemo {  
  
    public static void main(String[] args) {  
        var executor : ScheduledExecutorService = Executors.newScheduledThreadPool( corePoolSize: 1);  
        System.out.printf("Main: 开始于 %s\n", LocalTime.now());  
        // 执行5个任务  
        for (int i = 0; i < 5; i++) {  
            ScheduledTask task = new ScheduledTask( name: "Task " + i);  
            executor.schedule(task, delay: i + 1, TimeUnit.SECONDS);  
        }  
        // 关闭线程池  
        executor.shutdown();  
        // 等待线程池的中止  
        try {  
            executor.awaitTermination( timeout: 1, TimeUnit.DAYS);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        System.out.printf("线程池于: %s 正式关闭。", LocalTime.now());  
    }  
}
```

Run: ScheduleTaskDemo x

```
"C:\Program Files\Java\jdk-17\bin\java.exe"  
Main: 开始于 10:35:40.701380900  
Task 0: 开始于 : 10:35:41.716254100  
Task 1: 开始于 : 10:35:42.713283900  
Task 2: 开始于 : 10:35:43.726862700  
Task 3: 开始于 : 10:35:44.720005800  
Task 4: 开始于 : 10:35:45.713371800  
线程池于: 10:35:45.713371800 正式关闭。  
Process finished with exit code 0
```

ScheduleTaskDemo.java

补充知识：线程池的状态



状态名	说明
RUNNING	接收新任务并且处理阻塞队列里的任务
SHUTDOWN	拒绝新任务但是处理阻塞队列里的任务
STOP	拒绝新任务并且抛弃阻塞队列里的任务，同时中断正在处理的任 务。
TIDYING	所有任务都执行完（包含阻塞队列里面任务），当前线程池活动 线程为0，将要调用terminated方法。
TERMINATED	终止状态。terminated方法调用完成以后的状态。